ADVANCED PYTHON PROGRAMMING

(CSI3007)

NAME: ANEESH J.V.R
REG NO.: 23MID0306
SLOT: L31+L32+L49+L50

PROJECT REVIEW 2

Version 1:

https://drive.google.com/file/d/1RpL_Bqbvk6ws7Uqq2HWv3H9mGtSZhDxl/view?usp=drivesdk

Version 2:

https://drive.google.com/file/d/1uieSerQwWAaSZb97k-wDAammSRAQfNnN/view?usp=drivesdk

Transcript:

So, my project is a **Movie Analytics Dashboard**, and the main idea behind it is to make movie data easier to understand and analyze.

Let's say we have thousands of movies, each with different genres, release years, ratings, and other details. Just having all this data doesn't really help unless we can extract something useful from it.

For example, a content analyst might want to know which genres have higher ratings, or compare movies released in different years. Going through thousands of records manually would take a lot of time.

So, that's where my project comes in. It brings the data together in one place and presents it through visualizations, tables, and filters, making it easier for users to explore the data and identify patterns.

From a business perspective, this can help with content analysis, comparing movie performance based on available ratings, and supporting data-driven decisions. Of course, to predict actual commercial success, we would need additional information like revenue and viewership.

Now, coming to the technical side, I've built the application using Python, and it has three main parts: the frontend, backend, and database.

The **frontend** is developed in `frontend.py`. This is the part users interact with, where they can view the dashboard and explore the available information.

The **backend** uses FastAPI. The `main.py` file starts the application, and `api.py` defines the API endpoints. These endpoints allow the application to request, retrieve, and update data. I can also test them using FastAPI's interactive documentation.

Finally, for the **database**, I'm using SQLite for local development. The movie records are initially imported from CSV files and stored in the database. Currently, it contains around 4,800 movie records.

So, when a user requests information, the application processes that request, retrieves the relevant data, and displays the results on the dashboard. Any changes saved to the database can also persist when the application is restarted.

One thing I'm looking at as a future improvement is connecting the application to a shared PostgreSQL database and deploying the backend. This would allow multiple dashboards, including my partner's project, to access the same data through API endpoints instead of maintaining separate datasets.

Overall, the goal of this project is to turn raw movie data into something more useful and accessible, while understanding how the frontend, backend, APIs, and database work together in a real-world application.

Thank you.

MOVIE ANALYTICS DASHBOARD

Project Report

1. Introduction

The Movie Analytics Dashboard is a Python-based web application designed to make movie data easier to explore, understand, and analyze. It presents movie records through visualizations, tables, and interactive controls. The application combines a frontend interface, a backend API, and a database to provide an organized way of accessing and analyzing movie information.

2. Problem Statement

Movie datasets contain large amounts of information, including movie titles, genres, release years, and ratings. Manually reviewing thousands of records can be time-consuming and make it difficult to identify meaningful patterns. The Movie Analytics Dashboard addresses this problem by presenting the available data in a visual and accessible format, helping users explore records and compare information more efficiently.

3. Objectives

- To develop an interactive dashboard for exploring movie data.
- To present movie information through visualizations and tables.
- To support data retrieval and modification through backend API endpoints.
- To organize movie records using a database.
- To demonstrate the integration of frontend development, backend processing, and data storage in a web application.

4. Technology Stack

The project uses the following technologies:

- Programming Language: Python
- Backend Framework: FastAPI
- Application Server: Uvicorn
- Frontend: Python-based dashboard interface (`frontend.py`)
- Database: SQLite for local development
- Data Source: CSV files
- Supporting Libraries: Pandas, NumPy, SQLAlchemy, and other dependencies listed in `requirements.txt`

5. System Architecture

The application follows a layered architecture consisting of three major components.

Frontend: The frontend is implemented in `frontend.py`. It provides the user interface through which users can explore movie records, view visualizations, and interact with available filters and controls.

Backend: The backend uses FastAPI to handle application requests and expose API endpoints. The `main.py` file initializes the application, while `api.py` defines the endpoints. The `analytics.py` module supports analytical calculations, and other modules manage authentication and data validation.

Database: The application uses SQLite for local data storage. The `database.py` and `models.py` modules support database operations and data structures. Movie records are initially imported from CSV files using the project's data-loading and management modules.

6. Working of the Application

The application begins by importing movie records from CSV files into the database. The backend retrieves and processes the required information when the application requests it. The results are then presented through the dashboard interface.

The application also provides API documentation through FastAPI, allowing developers to inspect and test the available endpoints.

The local database currently contains approximately 4,803 movie records. Saved database changes can persist across application restarts, provided the database file is preserved.

7. Business Value and Applications

The dashboard helps users explore movie information without manually reviewing large spreadsheets. Its potential business applications include:

- Content Analysis: Exploring movie genres and release periods.
- Rating Analysis: Comparing movies based on available ratings.
- Data Exploration: Searching and filtering movie records efficiently.
- Decision Support: Helping analysts identify patterns and generate insights from available movie data.

The dashboard supports data-driven exploration. Assessing actual commercial success would require additional information, such as box-office revenue, streaming viewership, and audience demographics.

8. Current Implementation Status

The application has been successfully executed in a local development environment. The FastAPI server starts using Uvicorn and connects to the SQLite database containing approximately 4,803 movie records.

The project uses a Python virtual environment to manage dependencies. The API documentation is accessible through the `/docs` endpoint, enabling interactive testing of the available API operations.

9. Future Scope

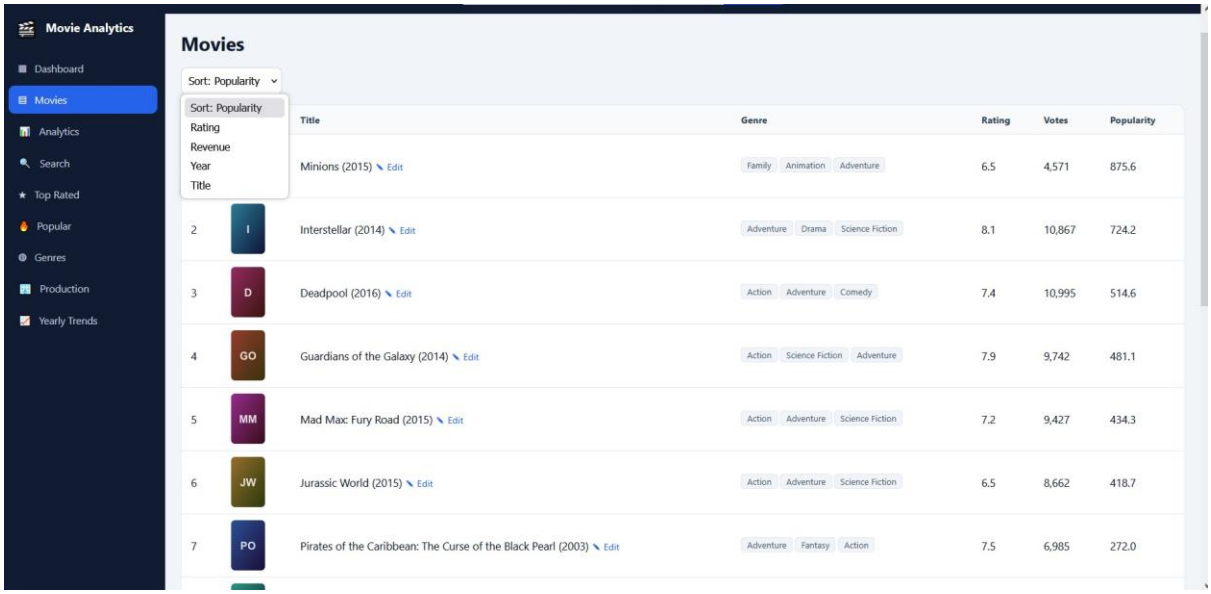
Future improvements include deploying the backend and connecting it to a shared PostgreSQL database. This would allow multiple dashboard applications to access a common dataset through API endpoints.

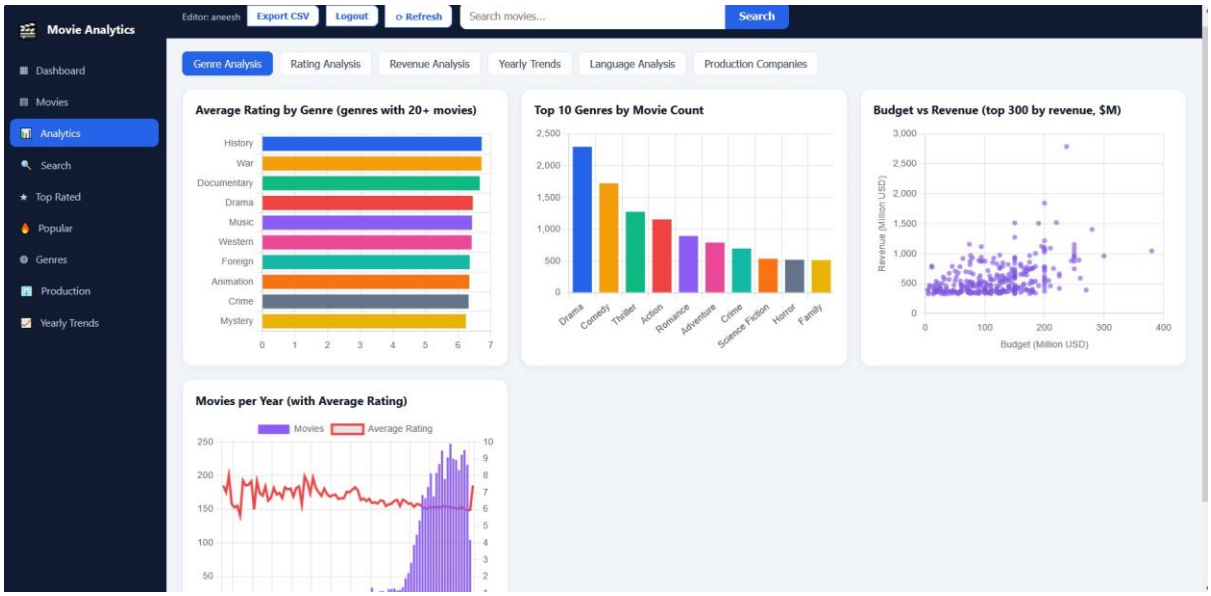
Additional improvements could include integrating more data sources, introducing advanced analytical metrics, and developing predictive features using suitable datasets.

10. Conclusion

The Movie Analytics Dashboard demonstrates how raw movie data can be organized, stored, processed, and presented through a web application. By combining Python, FastAPI, and SQLite, the project provides a practical example of full-stack application development and data analytics. It helps make movie data more accessible and supports more efficient exploration and analysis.




Movie Analytics
Dashboard
Movies
Analytics
Search
Top Rated
Popular
Genres
Production
Yearly Trends
Movies
Sort: Popularity
Sort: Popularity
Rating
Revenue
Year
Title
Title
Genre
Rating
Votes
Popularity
Minions (2015) Edit
Family
Animation
Adventure
6.5
4,571
875.6
2
Interstellar (2014) Edit
Adventure
Drama
Science Fiction
8.1
10,867
724.2
3
Deadpool (2016) Edit
Action
Adventure
Comedy
7.4
10,995
514.6
4
Guardians of the Galaxy (2014) Edit
Action
Science Fiction
Adventure
7.9
9,742
481.1
5
Mad Max: Fury Road (2015) Edit
Action
Adventure
Science Fiction
7.2
9,427
434.3
6
Jurassic World (2015) Edit
Action
Adventure
Science Fiction
6.5
8,662
418.7
7
Pirates of the Caribbean: The Curse of the Black Pearl (2003) Edit
Adventure
Fantasy
Action
7.5
6,985
272.0


Movie Analytics
Editor: aneesh
Export CSV
Logout
Refresh
Search movies...
Search
Genre Analysis
Rating Analysis
Revenue Analysis
Yearly Trends
Language Analysis
Production Companies
Average Rating by Genre (genres with 20+ movies)
History
War
Documentary
Drama
Music
Western
Foreign
Animation
Crime
Mystery
Top 10 Genres by Movie Count
Drama
Comedy
Thriller
Action
Romance
Adventure
Crime
Science Fiction
Horror
Family
Budget vs Revenue (top 300 by revenue, $M)
Revenue (Million USD)
Budget (Million USD)
Movies per Year (with Average Rating)
Movies
Average Rating


Movie Analytics
Editor: aneesh
Export CSV
Logout
Refresh
Search movies...
Search
Genre Analysis
Rating Analysis
Revenue Analysis
Yearly Trends
Language Analysis
Production Companies
Movies Released Per Year
Average Rating Per Year
Total Revenue Per Year ($B)